

AMATH 482: HOMEWORK 4

ROHAN PANDEY

University of Washington, Seattle, WA
rpande@uw.edu

ABSTRACT. This assignment explores Fully Connected Neural Networks (FCNs) and Deep Neural Networks (DNNs) for image classification using the FashionMNIST dataset. The dataset has 60,000 training images and 10,000 test images, each of size 28x28 pixels.

We performed hyperparameter tuning, comparing different optimizers (SGD, Adam, RMSProp), weight initialization methods (Xavier Normal, Kaiming (He) Uniform, Random Normal), and regularization techniques such as Dropout and Batch Normalization. Our experiments analyzed model convergence, overfitting, and generalization.

The final results include training loss curves, accuracy trends, and insights into the most effective configurations for FashionMNIST classification.

1. INTRODUCTION AND OVERVIEW

Image classification is one of the fundamental tasks in computer vision and has experienced significant advancements with the advent of deep learning. Deep neural networks, by learning hierarchical representations of data, have outperformed traditional methods in various applications. In this assignment, we explore the use of Fully Connected Neural Networks (FCNs) for classifying images from the FashionMNIST dataset, a modern and more challenging alternative to the classic MNIST dataset [4].

2. THEORETICAL BACKGROUND

Fully Connected Neural Networks (FCNs) consist of layers that perform linear transformations followed by non-linear activations to learn hierarchical representations of data. In each layer, the network computes a weighted sum of its inputs, adds a bias, and then applies a non-linear function (commonly the Rectified Linear Unit, ReLU) to capture complex features. The ReLU activation function is also defined below. For classification tasks, the final layer outputs a vector of raw scores (logits), which are then converted into a probability distribution using the softmax function:

$$(1) \quad \text{ReLU}(x) = \max(0, x)$$

$$(2) \quad \sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}},$$

where C denotes the number of classes [1]. This transformation allows the network to produce interpretable probabilities for each class.

The performance of the network is evaluated using the Cross-Entropy Loss, which measures the dissimilarity between the predicted probability distribution and the true one-hot encoded labels:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\sigma(\mathbf{z})_i),$$

ensuring that lower loss values correspond to predictions that are closer to the true labels [1].

To improve generalization and mitigate overfitting, regularization techniques such as **dropout** are employed. Dropout randomly deactivates a fraction of neurons during training, which forces the network to learn redundant representations and reduces reliance on any single neuron [3]. Additionally, proper weight initialization is crucial for stable training. Methods like Xavier (Glorot) initialization and Kaiming (He) initialization are designed to maintain consistent variance of activations across layers, thereby helping to prevent the vanishing or exploding gradient problem [2].

Together, these concepts—layer transformations with non-linear activations, probability mapping via softmax, error quantification using cross-entropy loss, dropout regularization, and robust weight initialization—form the foundation for training effective FCNs for image classification tasks.

3. ALGORITHM IMPLEMENTATION AND DEVELOPMENT

Below is the high-level algorithm implementation for the tasks performed in this assignment.

Algorithm 1 FCN Training and Hyperparameter Tuning for FashionMNIST

```

1: Input: FashionMNIST dataset, hyperparameters: learning_rate, epochs, batch_size, hidden_layers
2: Output: Trained model and performance metrics
3: /* Data Loading and Preprocessing */
4: Load FashionMNIST dataset, normalize, split into Training, Validation, and Test sets.
5: Reshape images to 784-dimensional vectors.
6: /* Task 2: Baseline Model Training */
7: Initialize FCN model with hidden layers (e.g., [128, 64]), Cross-Entropy loss, and SGD optimizer.
8: for epoch = 1 to epochs do
9:     Train model on batches, update weights via backpropagation.
10:    Compute training loss and validation accuracy.
11: end for
12: Evaluate final model on Test set.
13: /* Task 3: Hyperparameter Tuning */
14: for optimizer  $\in$  {SGD, RMSProp, Adam}, dropout_rate  $\in$  {0.0, 0.5}, init_method  $\in$  {Random, Xavier, Kaiming} do
15:     Initialize FCN with corresponding settings, train, and evaluate.
16: end for
17: Train FCN with Batch Normalization.
18: /* Results and Reporting */
19: Plot loss and accuracy curves, summarize test accuracies across experiments.

```

4. COMPUTATIONAL RESULTS

First we can talk about the baseline model. This was after finding a working configuration of the FCN model that has an adjustable number of hidden layers and neurons in each layer with relu activation. Recall that FCN stands for Fully Connected Networks. The loss curve as well as its validation accuracy is shown below in Figure 1.

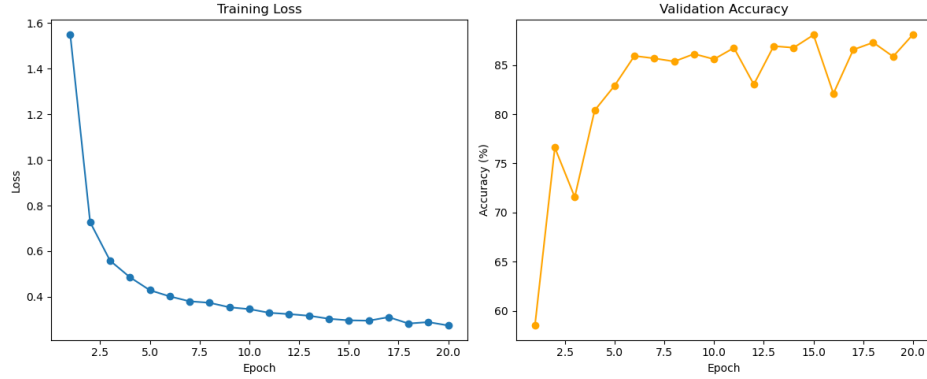


FIGURE 1. FCN Baseline Model: Loss and Validation Accuracy with Test Accuracy of 87.03%

The way I was able to achieve this accuracy was by increasing epochs as well as learning rate, although I strongly believe that epochs was more harmful than helpful as we can see around epoch 12 and 15 the accuracy dropped suddenly. This is a sign of overfitting.

4.1. Hyperparameter Tuning.

4.1.1. *Optimizers.* We then worked on tuning our parameters on the baseline model, using RMSProp, Adam and SGF optimizers. The results of this tuning are shown below, we have the plots of loss curves, and also a table detailing the test accuracies. Note the legend on the plots to see which optimizer is which.

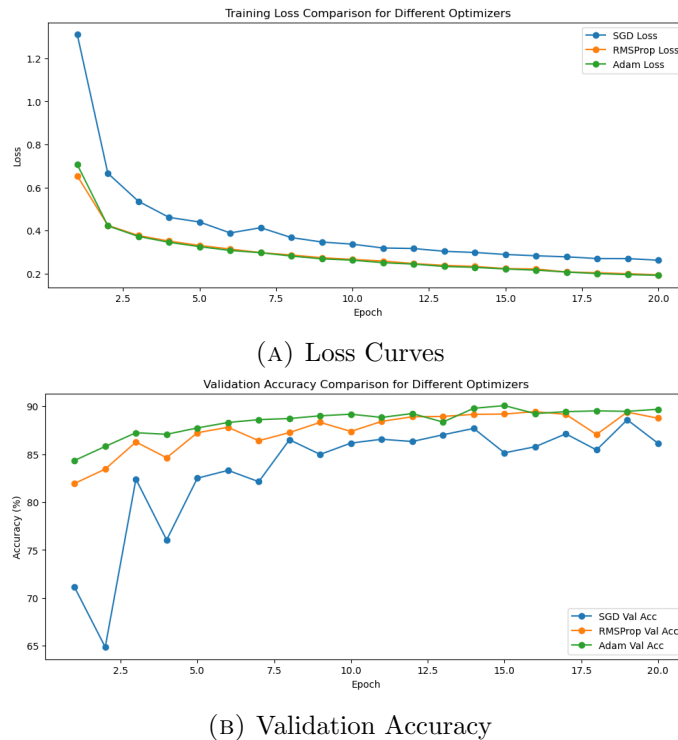


FIGURE 2. FCN Optimizer Tuning: Loss and Validation Accuracy

Optimizer	Test Accuracy (%)
SGD	85.26
RMSProp	87.34
Adam	88.52

TABLE 1. Test Accuracies for Different Optimizers

From 1, we can see that Adam is the best optimizer for this model, with a test accuracy of 88.52%. This is followed by RMSProp and then SGD. I think this might be because Adam is a combination of RMSProp and Momentum, which allows it to converge faster and more efficiently.

4.1.2. *Regularization.* We then used Dropout regularization to prevent overfitting. The results of this tuning are shown below, we have the plots of loss curves, and also a table detailing the test accuracies. Note the legend on the plots to see which dropout rates we used and what the results were.

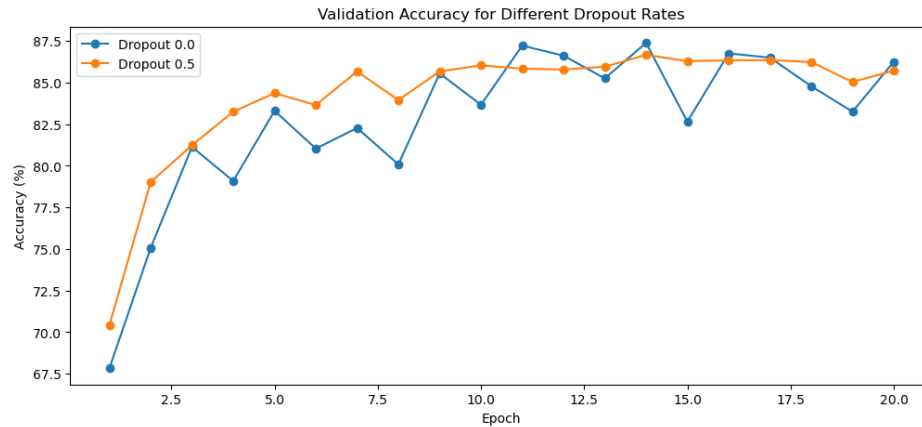


FIGURE 3. FCN Dropout Regularization: Loss and Validation Accuracy

Dropout Rate	Test Accuracy (%)
0.0	85.26
0.5	84.59

TABLE 2. Test Accuracies for Different Dropout Rates

From 2, we can see that Dropout regularization did not help our model. This is because Dropout is a technique that randomly sets a fraction of input units to zero at each update during training time, which usually helps with overfitting. However, in this case, it seems to have made the model worse.

4.1.3. *Initialization.* We then used different weight initialization methods such as Random Normal, Xavier Normal, and Kaiming (He) Uniform. The results of this tuning are shown below, we have the plots of loss curves, and also a table detailing the test accuracies. Note the legend on the plot as we have plotted all the weight initialization methods on the same plot.

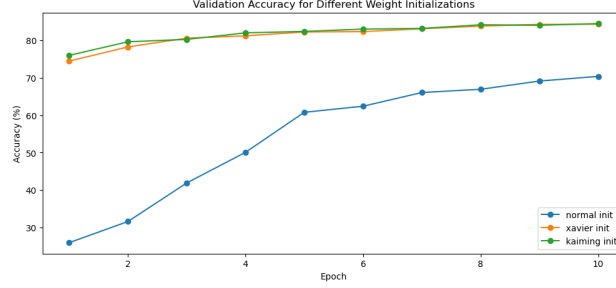


FIGURE 4. FCN Weight Initialization: Loss and Validation Accuracy

Initialization Method	Test Accuracy (%)
Random Normal	69.40
Xavier Normal	83.18
Kaiming (He) Uniform	83.68

TABLE 3. Test Accuracies for Different Weight Initialization Methods

From 3, we can see that Kaiming (He) Uniform initialization is the best weight initialization method for this model, with a test accuracy of 83.68%. This is followed by Xavier Normal and then Random Normal. This is because Kaiming (He) Uniform initialization is specifically designed for ReLU activation functions, which we are using in our model. Alternatively, something that I found through testing was that the Kaiming initializer was not learning at all if the learning rate was too high. Originally it was set to 0.1, and I had to reduce it to 0.01 to get the model to learn otherwise the test accuracy was stuck at 10% for all 20 epochs, as an extra precaution I also reduced the number of epochs to 10 to prevent overfitting.

4.1.4. *Normalization.* Lastly, we included Batch Normalization in our model. In addition to the training loss curve and the validation accuracy achieved by the normalization techniques, I also plotted the Baseline models loss curve and validation accuracy for comparison. The results are shown below.

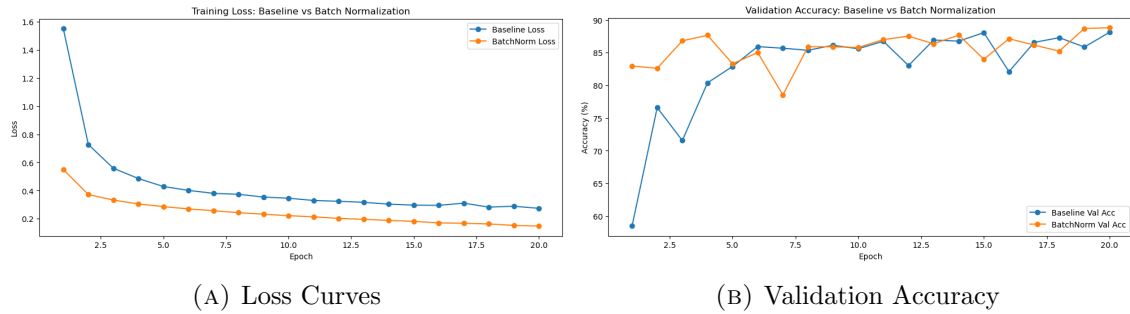


FIGURE 5. FCN Normalization Techniques: Loss and Validation Accuracy of 88.01%

Clearly we can see that the baseline loss curve is greater than the loss curve of the Batch Normal loss curve. Similarly, the validation accuracy of the baseline model is more scattered in comparison to the Batch Normal which has a bit of variance in the initial epochs but then stabilizes. The test accuracy of the Batch Normal model is 88.01% which is better than the baseline model. This is

because Batch Normalization normalizes the output of a previous activation layer by subtracting the batch mean and dividing by the batch standard deviation. This has the effect of stabilizing and accelerating the learning process, and also allows for higher learning rates.

5. SUMMARY AND CONCLUSIONS

In this assignment, we explored Fully Connected Neural Networks (FCNs) for image classification using the FashionMNIST dataset. We implemented a baseline model and performed hyperparameter tuning to optimize the model's performance. Our experiments included comparing different optimizers, weight initialization methods, regularization techniques, and normalization methods.

Our results showed that Adam optimizer outperformed SGD and RMSProp, achieving a test accuracy of 88.52%. We also found that Kaiming (He) Uniform initialization was the most effective weight initialization method, with a test accuracy of 83.68%. Dropout regularization did not improve the model's performance, while Batch Normalization significantly enhanced the model's accuracy to 88.01%.

In conclusion, our experiments demonstrated the importance of hyperparameter tuning and the impact of different optimization techniques on model performance. Future work could involve exploring more advanced architectures such as Convolutional Neural Networks (CNNs) for image classification tasks, as well as investigating the effects of other regularization techniques and normalization methods on model performance.

ACKNOWLEDGEMENTS

The author is thankful to Professor Frank for providing the materials and resources needed to complete this assignment. The author is also thankful to the peers Eric Ye and Chenab and Brighton for collaborative work and brainstorm sessions for the Theoretical Background section and visualizations in Python.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [2] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [4] H. Xiao, K. Rasul, and R. Vollgraf. Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms, 2017.